

Web Systems Development

Week 4 · CSS: the cascade, the tools, the frameworks



September 15, 2026

AGENDA

- Noting responsive design strategies
- Questions from CSS Exercises
- What do we mean by DOM

Flex puts the boxes in a row

THE REQUEST

Set the d1 container to display its three boxes side by side.

WHAT IT DEMONSTRATES

flex creates a flex container. Its default row direction places direct children along a horizontal main axis in this page.

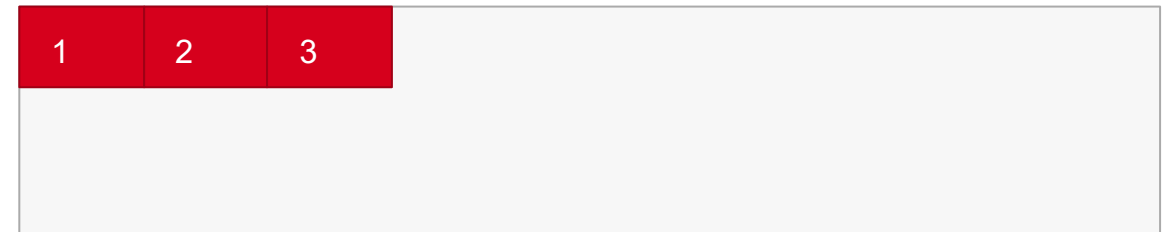
EXPECTED RESULT

The three boxes form one row. They can touch until you add spacing.

TAILWIND CLASSES

```
<div id="d1" class="flex">  
  <!-- Keep the three boxes -->  
</div>
```

LAYOUT PREVIEW



UNDERLYING CSS

```
display: flex;
```

The row sits in the center

THE REQUEST

Keep the boxes in a row and group all three in the middle of d2.

WHAT IT DEMONSTRATES

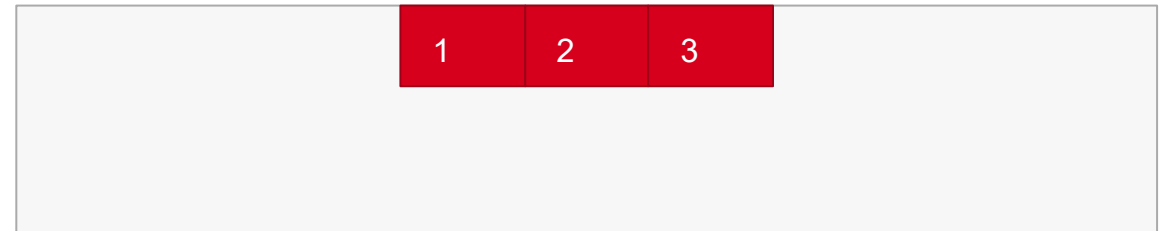
justify-center aligns the group on the main axis. With the default row direction, that means horizontal centering.

EXPECTED RESULT The group has equal unused space to its left and right.

TAILWIND CLASSES

class="flex justify-center"

LAYOUT PREVIEW



UNDERLYING CSS

```
display: flex;  
justify-content: center;
```

The outside boxes reach both ends

THE REQUEST

Place the first box at the left edge and the last at the right edge, with the third between them.

WHAT IT DEMONSTRATES

`justify-between` distributes the available space between items. It adds no distribution space before the first or after the last.

EXPECTED RESULT

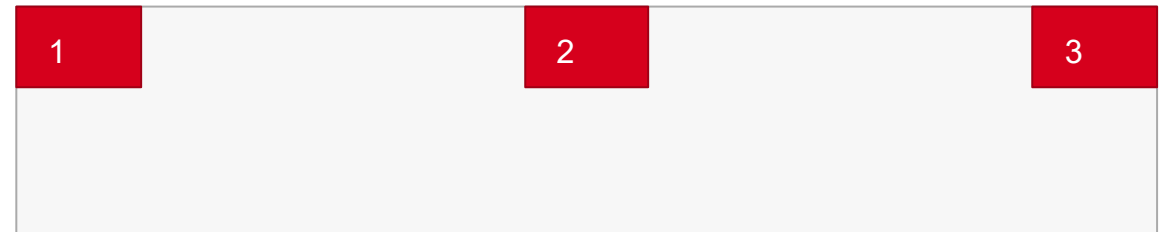
The outside boxes sit at the container's content edges. The middle box sits between them.

[Tailwind docs: justify-content](#)

TAILWIND CLASSES

`class="flex justify-between"`

LAYOUT PREVIEW



UNDERLYING CSS

```
display: flex;  
justify-content: space-between;
```

Each box gets space on both sides

THE REQUEST

Give every box equal space around it, including space at the ends of the row.

WHAT IT DEMONSTRATES

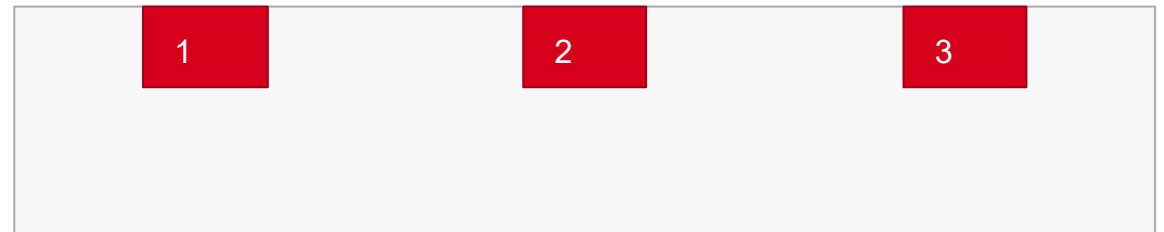
justify-around allocates equal space on each side of an item. Interior gaps combine two shares, so they are twice the edge spaces.

EXPECTED RESULT The row has space at both edges, with larger gaps between neighboring boxes.

TAILWIND CLASSES

class="flex justify-around"

LAYOUT PREVIEW



UNDERLYING CSS

```
display: flex;  
justify-content: space-around;
```

Different heights share a middle line

THE REQUEST

Add items-center to the flex h-full container in d5.

WHAT IT DEMONSTRATES

items-center aligns children on the cross axis. In this row, the cross axis is vertical, so the boxes share a vertical midpoint.

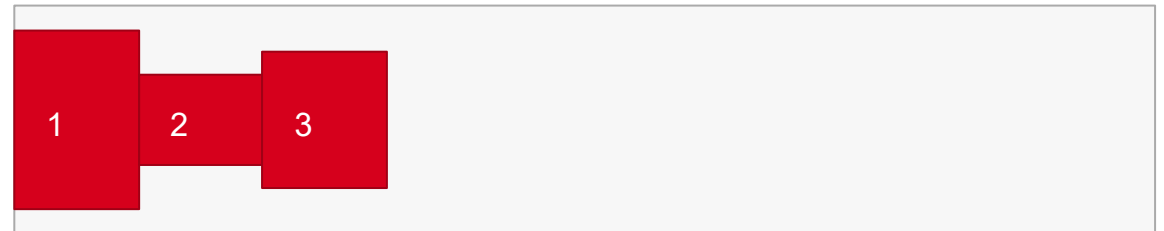
EXPECTED RESULT

The tall, short and medium boxes line up through their centers.

TAILWIND CLASSES

```
class="flex h-full items-center"
```

LAYOUT PREVIEW



UNDERLYING CSS

```
display: flex;  
height: 100%;  
align-items: center;
```

Gap separates neighboring items

THE REQUEST

Separate the boxes in d6 without adding a margin to each box.

WHAT IT DEMONSTRATES

gap-4 sets spacing on the parent. In the default theme, it gives neighboring flex items a 1rem gap.

EXPECTED RESULT

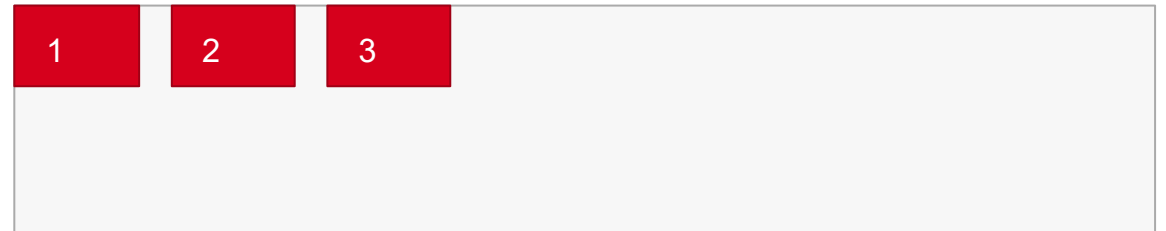
Equal gaps appear between the boxes. Gap adds no padding around the outside of the row.

[Tailwind docs: gap](#)

TAILWIND CLASSES

```
class="flex gap-4"
```

LAYOUT PREVIEW



UNDERLYING CSS

```
display: flex;  
gap: 1rem;
```

The main axis becomes vertical

THE REQUEST

Keep a flex container and stack its boxes from top to bottom with flex-col.

WHAT IT DEMONSTRATES

flex-col changes flex-direction to column. It makes the main axis vertical and the cross axis horizontal in this page.

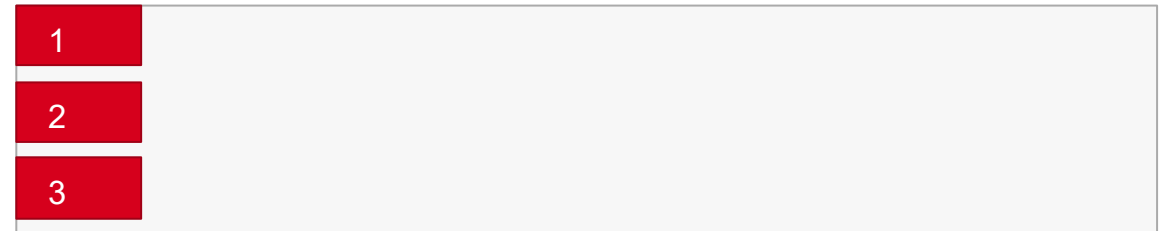
EXPECTED RESULT

The boxes form a column. gap-2 leaves a small space between them.

TAILWIND CLASSES

```
class="flex flex-col gap-2"
```

LAYOUT PREVIEW



UNDERLYING CSS

```
display: flex;  
flex-direction: column;  
gap: 0.5rem;
```

Items-center now centers horizontally

THE REQUEST

Keep d8 as a column and add items-center to center the boxes horizontally.

WHAT IT DEMONSTRATES

items-* always controls the cross axis. Changing to flex-col makes that axis horizontal in this layout.

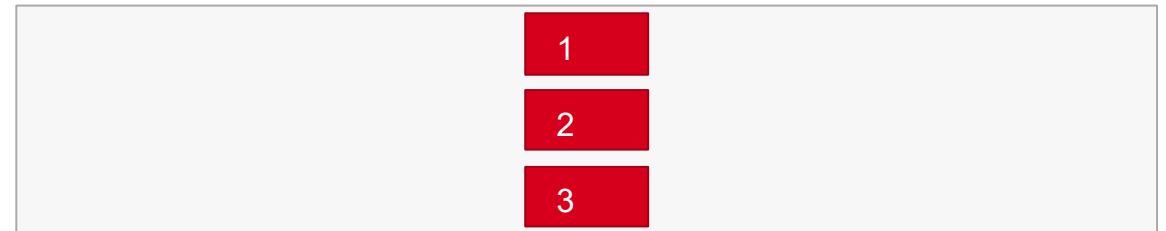
EXPECTED RESULT

The column sits in the horizontal center. The existing justify-center also centers it vertically.

TAILWIND CLASSES

```
class="flex flex-col gap-2 h-full  
justify-center items-center"
```

LAYOUT PREVIEW



UNDERLYING CSS

```
flex-direction: column;  
align-items: center;
```

One box is centered on both axes

THE REQUEST

Add justify-center and items-center to the flex h-full container in d9.

WHAT IT DEMONSTRATES

justify-center handles the main axis and items-center handles the cross axis. h-full supplies the vertical space for centering.

EXPECTED RESULT

The single box sits at the center of the inner container, horizontally and vertically.

TAILWIND CLASSES

```
class="flex h-full  
      justify-center items-center"
```

LAYOUT PREVIEW



UNDERLYING CSS

```
justify-content: center;  
align-items: center;
```

Six boxes fill three equal columns

THE REQUEST

Add grid and grid-cols-3 to g1. Keep the six child boxes.

WHAT IT DEMONSTRATES

grid establishes a grid container. grid-cols-3 defines three equal tracks, and auto-placement puts the remaining items in a new row.

EXPECTED RESULT

Six boxes form three columns and two rows. They touch because this drill has no gap yet.

[Tailwind docs: grid-template-columns](#)

TAILWIND CLASSES

class="grid grid-cols-3"

LAYOUT PREVIEW

1	2	3
4	5	6

UNDERLYING CSS

```
display: grid;  
grid-template-columns:  
  repeat(3, minmax(0, 1fr));
```

One gap utility spaces both directions

THE REQUEST

Add gap-4 to the existing grid grid-cols-3 container in g2.

WHAT IT DEMONSTRATES

In a grid, gap sets both the row gap and the column gap. The container controls spacing for all of its cells.

EXPECTED RESULT

The six boxes have equal spacing across the columns and between the rows.

TAILWIND CLASSES

```
class="grid grid-cols-3 gap-4"
```

LAYOUT PREVIEW

1	2	3
4	5	6

UNDERLYING CSS

```
gap: 1rem;  
/* row gap and column gap */
```

Two columns produce three rows

THE REQUEST

Set g3 to grid-cols-2 while preserving its grid and gap-4 classes.

WHAT IT DEMONSTRATES

The column count belongs to the container. Changing it rearranges the same six children without changing their HTML.

EXPECTED RESULT

The same six boxes form two columns and three rows, with the gaps preserved.

TAILWIND CLASSES

```
class="grid grid-cols-2 gap-4"
```

LAYOUT PREVIEW

1	2
3	4
5	6

UNDERLYING CSS

```
grid-template-columns:  
  repeat(2, minmax(0, 1fr));
```

One item spans two columns

THE REQUEST

Add col-span-2 to the dark box g4. Keep its parent as a three-column grid.

WHAT IT DEMONSTRATES

A span controls an individual grid item. The container defines the tracks, while the child selects how many it occupies.

EXPECTED RESULT

The dark box covers two column tracks and their intervening gap. The next box fills the remaining track.

[Tailwind docs: grid-column](#)

TAILWIND CLASSES

```
<!-- On the dark box g4 -->  
class="bg-teal-900 h-12 rounded  
col-span-2"
```

LAYOUT PREVIEW



UNDERLYING CSS

grid-column: span 2 / span 2;

One item spans two rows

THE REQUEST

Add row-span-2 to the dark box g5. Leave the surrounding grid unchanged.

WHAT IT DEMONSTRATES

Row spans reserve vertical tracks for an item. The supplied dark box has no fixed height, so it can stretch across both rows.

EXPECTED RESULT

The dark item stretches across two rows while the four smaller boxes occupy the adjacent cells.

[Tailwind docs: grid-row](#)

TAILWIND CLASSES

```
<!-- On the dark box g5 -->  
class="bg-teal-900 rounded row-span-2"
```

LAYOUT PREVIEW



UNDERLYING CSS

```
grid-row: span 2 / span 2;
```

A breakpoint changes the column count

THE REQUEST

Make g6 one column below 768px and three columns at 768px and wider. Keep gap-4.

WHAT IT DEMONSTRATES

Unprefixed utilities establish the base layout. md: applies an override from the medium breakpoint upward.

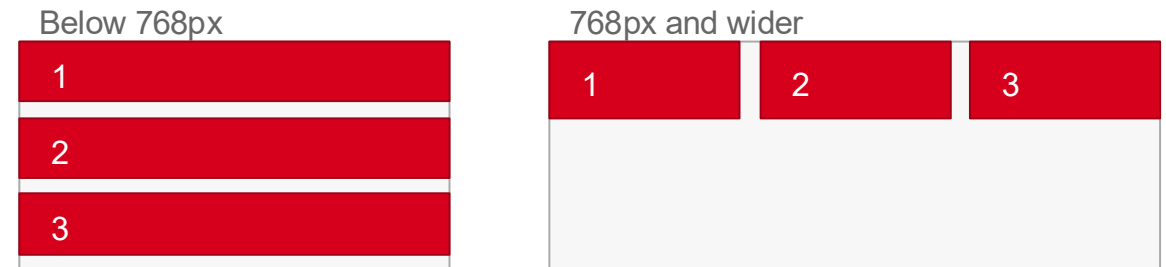
EXPECTED RESULT

The boxes stack below 768px and form a row of three at wider sizes. The check stays green on both sides.

[Tailwind docs: responsive-design](#)

TAILWIND CLASSES

```
class="grid grid-cols-1 gap-4  
md:grid-cols-3"
```



UNDERLYING CSS

Base: one column

At md and wider: three columns

The article spans two tracks on wide screens

THE REQUEST

Use one column by default. At md, use three columns and let the article span two.

WHAT IT DEMONSTRATES

Responsive classes belong on both levels: the parent changes its tracks, and the article changes its span at the same breakpoint.

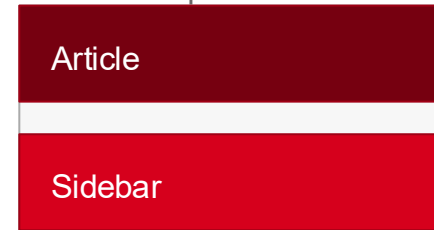
EXPECTED RESULT

Below 768px, article and sidebar stack. At wider sizes, the article takes two tracks and the sidebar one.

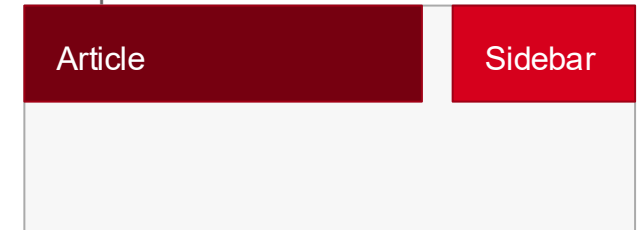
TAILWIND CLASSES

g7: grid grid-cols-1 gap-4 md:grid-cols-3
g7a: bg-teal-900 h-16 rounded
md:col-span-2

Below 768px



768px and wider



UNDERLYING CSS

At md: 3 parent tracks
Article: spans 2 tracks

The drills become the homework layout

Task 1: header and navigation

Header: flex justify-between items-center
Navigation: flex gap-4
Add padding and the requested styling.

Task 3: article and sidebar

Parent: grid grid-cols-1 gap-4
md:grid-cols-3
Article: md:col-span-2

Task 2: three responsive stages

grid grid-cols-1 gap-4
md:grid-cols-2 lg:grid-cols-3

375px: one column
800px: two columns
1280px: three columns

Grid drill 6 practices a simpler one-to-three change at md.

PAUSE AND DEFINE

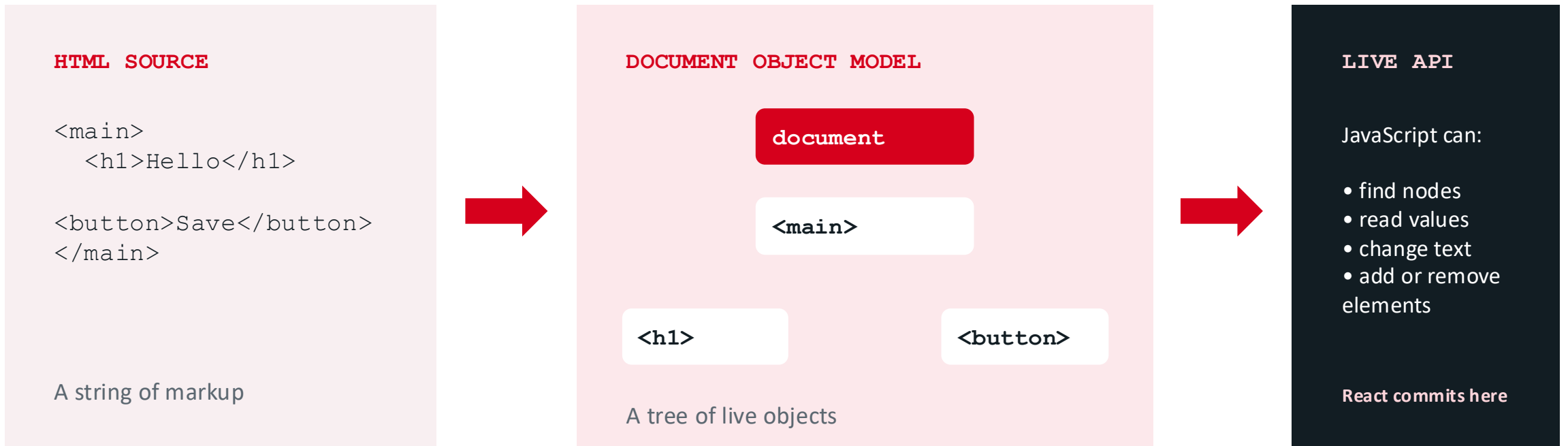
What do we mean by the DOM?

Before React can commit a change, what exactly is it changing?

THE ANSWER

The DOM is the browser's live object model of the page

The browser turns HTML into a tree of objects that JavaScript—and React—can read and change.



React does not edit your HTML file. It commits changes to this live DOM tree in the browser.

WHAT REACT IS

React is a JavaScript library for building UIs

A library, not magic

```
npm install react
```

Open source, released by Facebook in 2013 and maintained by Meta and a large community. It does one job: turn your data into what the browser shows.

Built from components

```
<TodoList items={data} />
```

A component is a JavaScript function that returns markup. You build a page by composing small components, like HTML tags you invented.

Declarative rendering

```
setState() = re-render
```

You never edit the page by hand. Describe what the screen should show for the current data, and React updates the DOM when that data changes.

A React app is just JavaScript functions that return markup, re-run automatically whenever the data changes.

React is still core JavaScript literacy

83.6%

of respondents have used React

#3 most-used library overall

Hands-on users rate React more positively than people who only know of it.

STATE OF JS 2025 • LIBRARIES BY USAGE



Learning React means practicing the patterns modern interfaces rely on: components, state, events, and data flow.

React code follows consistent naming conventions

Components and files

`UserProfile.jsx`

Components are PascalCase, matching their exported name: UserProfile, NavBar. The file is named to match the component it exports.

Props and handlers

`useAuth()` · `onClick` ·
`handleClick`

Props, variables, and functions are camelCase. Hooks are camelCase prefixed with use. Handler props start with on; the function itself starts with handle.

Booleans & constants

`isOpen` · `MAX_RETRIES`

Boolean props and state are prefixed is, has, or should. True constants are UPPER_SNAKE_CASE; config objects stay camelCase.

None of this is enforced by React itself, but every major style guide and linter (ESLint, Airbnb) expects it, so following it keeps code reviewable across a team.

The `/` makes it a **self-closing tag**. It means you're rendering `Product` without putting any content between opening and closing tags.

These are equivalent:

```
</> JavaScript
```



```
<Product name="Apples" />
```

```
</> JavaScript
```



```
<Product name="Apples"></Product>
```

WHERE THIS FITS

From classes to components

What you have

HTML + Tailwind

You can build any static layout: a header, a card grid, a sidebar that stacks on a phone. HW3 proved it. The page is exactly as correct as the file you typed.

What is missing

anything that changes

A counter, a filter, a form, a cart. Right now the only way to change what is on screen is to edit the file and reload.

What React adds

$UI = f(data)$

Write a function that turns data into markup. Change the data; React works out what on the page has to change and changes only that.

Nothing you learned about HTML or Tailwind is being replaced. React is a new way to produce the same elements and the same class names.

THE PROBLEM

Keeping the page in sync by hand

WHAT YOU WOULD WRITE TODAY

```
let count = 0;
const button = document.querySelector('#add');
const label = document.querySelector('#total');
const empty = document.querySelector('#empty-msg');

button.addEventListener('click', () => {
  count = count + 1;
  label.textContent = `Added ${count}`; // don't forget
  empty.hidden = count > 0;           // or this
  button.disabled = count >= 10;      // or this
});
```

WHAT GOES WRONG

- One piece of data, three places on screen that have to be told about it.
- Add a fourth thing that depends on count and you must remember to update it here.
- Two people edit this and one of them forgets. The page now disagrees with itself.
- The bug is invisible in the code: nothing is wrong, something is missing.

The data and the screen are two separate things that you are responsible for keeping equal. That is the job React takes away.

This is not a strawman — it is how the web was built for fifteen years, and it is how you would do it with what you know now.

THE IDEA

Throw it away and draw it again

What if, every time the data changed, you deleted the whole page and rebuilt it from scratch?

- Then there is no syncing to do. There is one description of the page, and it is always built from the current data.
- You could never forget to update something, because you never update anything — you rebuild.
- The three-places-to-remember problem stops existing, rather than getting easier.

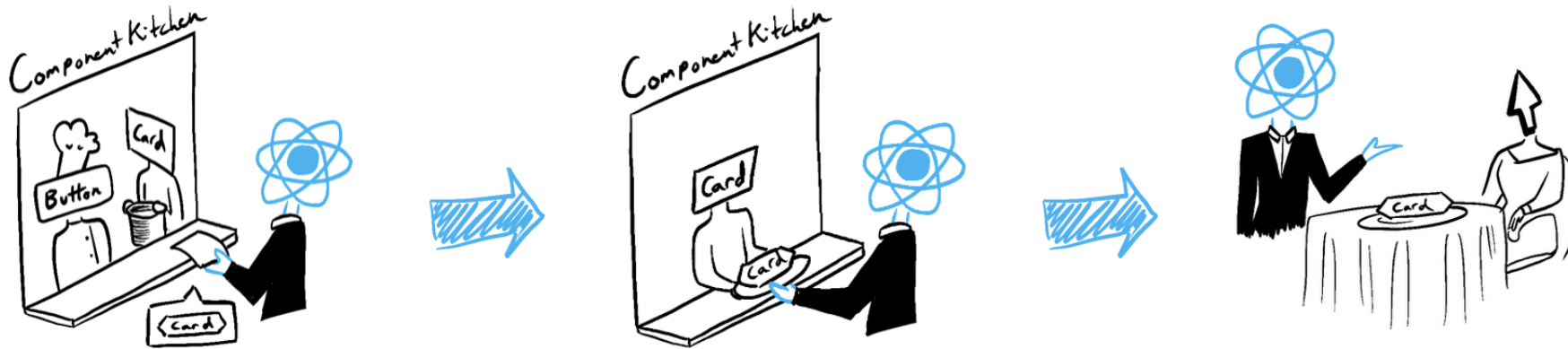
The obvious objection

Rebuilding the whole DOM on every keystroke would be unusably slow, and it would throw away scroll position, focus, and whatever you were typing.

Pete Hunt made this argument at JSConf EU in 2013 to a room that mostly did not believe him.

Requesting and serving UI has three steps

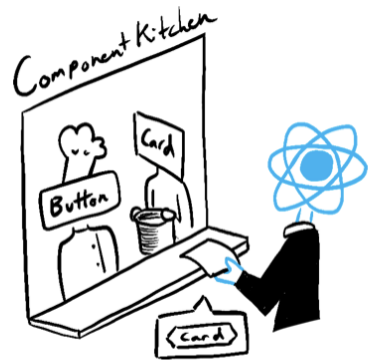
Trigger work. Call components to calculate the UI. Commit only the DOM changes needed.



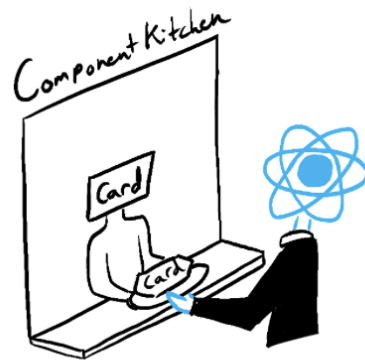
Step 1: Trigger a render

There are two reasons for a component to render:

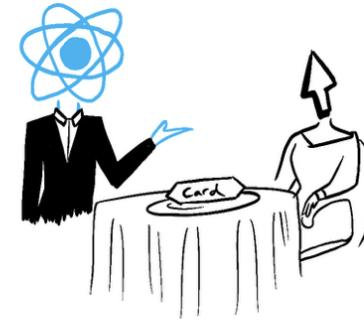
1. It's the component's **initial render**.
2. The component's (or one of its ancestors') **state has been updated**.



Trigger



Render

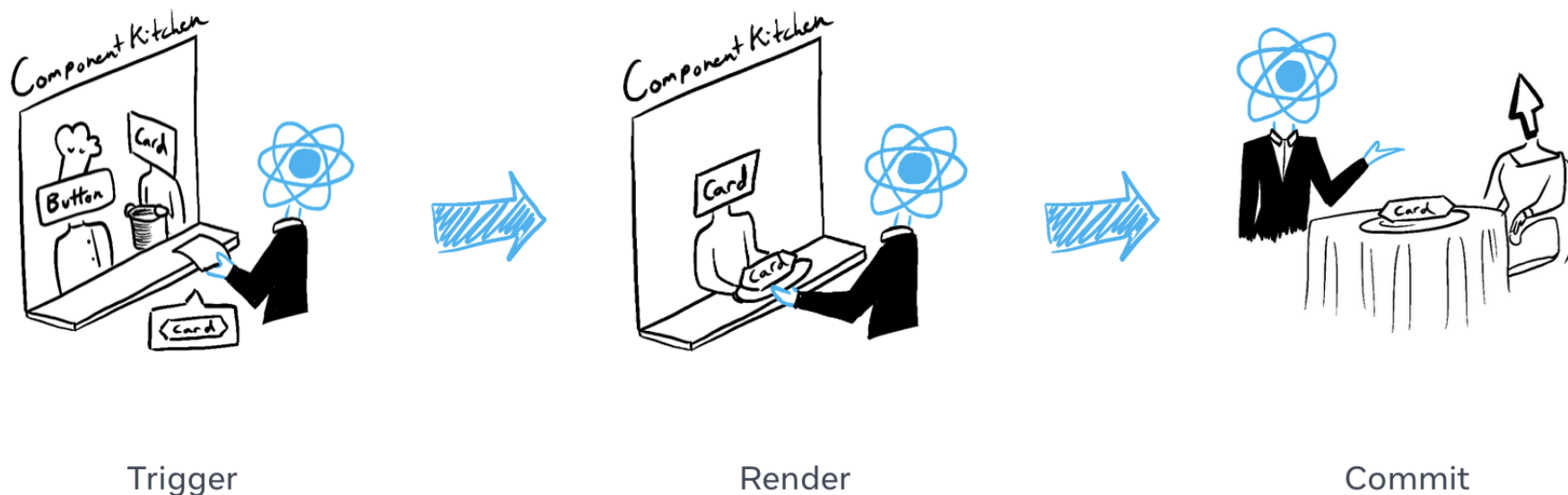


Commit

Step 2: React renders your components

After you trigger a render, React calls your components to figure out what to display on screen. **“Rendering” is React calling your components.**

- **On initial render**, React will call the root component.
- **For subsequent renders**, React will call the function component whose state update triggered the render.



Step 3: React commits changes to the DOM

After rendering (calling) your components, React will modify the DOM.

- **For the initial render**, React will use the `appendChild()` DOM API to put all the DOM nodes it has created on screen.
- **For re-renders**, React will apply the minimal necessary operations (calculated while rendering!) to make the DOM match the latest rendering output.

Epilogue: Browser paint

After rendering is done and React updated the DOM, the browser will repaint the screen. Although this process is known as “browser rendering”, we’ll refer to it as “painting” to avoid confusion throughout the docs.

Illustrated by [Rachel Lee Nabors](#)



THE UNIT

A component is a function

WITHOUT JSX — WHAT ACTUALLY RUNS

```
function ProductHeading() {  
  return createElement(  
    'h2',  
    { className: 'text-xl font-bold' },  
    'Pantry'  
  );  
}
```

WITH JSX — WHAT YOU WRITE

```
function ProductHeading() {  
  return <h2 className="text-xl font-bold">  
    Pantry  
  </h2>;  
}
```

These are the same program. JSX is not a template language and it is not HTML — it is shorthand that the build step turns into the call on the left.

- A component is an ordinary function. It takes data and returns a description.
- Its name starts with a capital letter — that is how JSX tells your component from a built-in h2.
- You call it by writing <ProductHeading /> somewhere, not ProductHeading().

Four rules that catch everyone

class becomes className

class is a reserved word in JavaScript. Every other Tailwind thing is identical: justify-between is still justify-between.

```
<div className="flex gap-4">
```

close every tag

`<input />`, `<br />`, `<img />`. HTML forgives you; JSX does not.

```
<input id="name" />
```

return one parent

A function returns one value, so JSX returns one element. Use `<> </>` when you do not want a real wrapper.

```
return <> <h2/> <p/> </>;
```

braces run JavaScript

Anything between tags is literal text until you open a brace.

```
<p>{product.name}</p>
```

Drills 1 to 3 are these four rules, one at a time, with the browser telling you when you have it.

PROPS

Props are inputs to a component

A parent passes values in. The component reads them and decides what to display.

1 PASS

```
<Product  
  name="Apples"  
  expired={false}  
>
```



2 READ

```
function  
Product(  
  { name,  
    expired }  
)
```



3 DISPLAY

Apples

Fresh

Props work like function arguments: they let the same component display different data.

Parent passes props → component reads props → UI changes

LISTS

Let the data decide the markup

```
const products = [  
  { id: 'a', name: 'Apples' },  
  { id: 'r', name: 'Rice' },  
  { id: 'b', name: 'Beans' },  
];  
  
<ul className="grid grid-cols-1 gap-4 md:grid-cols-3">  
  {products.map(product => (  
    <li key={product.id}>{product.name}</li>  
  ))}  
</ul>
```

- map turns an array of data into an array of elements. One template, any number of items.
- Add a fourth product to the array and the page grows. You write no new markup.
- key tells React which rendered item goes with which piece of data when things move or disappear.
- Use the id from your data. Never the array index, never Math.random().

The Tailwind grid classes are exactly HW3's. React decides how many cards exist; Tailwind still decides how they sit.

Use one component for every task

```
const tasks = [
  { id: 1, title: 'Read', done: true },
  { id: 2, title: 'Practice', done: false },
  { id: 3, title: 'Submit', done: false },
];

<ul>
  {tasks.map(task => (
    <Task key={task.id}
      title={task.title}
      done={task.done} />
  ))}
</ul>
```

- map runs once for each task and returns one Task element.
- Each task supplies different props, so the same component displays different content.
- Add, remove, or reorder the data and the rendered list follows automatically.
- key still comes from the stable id in the data.

map controls how many components React creates; props control what each component displays.

Choosing what to show

A value, either way

```
{expired ? 'Expired' : 'Fresh'}
```

A ternary is an expression, so it can sit inside braces. Use it when there are two things and you want one.

Sometimes nothing

```
{count > 0 && <Badge n={count} />}
```

Renders the badge when the left side is true, nothing when it is false.
Careful: `0 && ...` renders a literal 0.

- There is no if in JSX, because if is a statement and JSX holds expressions. An if above the return is fine, and often clearer.
- Two whole Tailwind class strings chosen by a ternary is a normal and readable thing to do.

If a condition is getting long, pull it into a variable above the return and put the variable in the braces.

What you can build already

1 A component

a function that returns a description of some UI

2 Props

the arguments that make one definition serve many instances

3 map + key

data decides how many elements exist

4 A ternary

data decides which of two things appears

Everything so far is a one-way trip: data in, markup out. Nothing on the page can change yet, because nothing remembers anything. That is Friday.

Drills 1 to 6 cover all four. Do them in class; finish any that are left before Friday.

STATE

Memory that survives a re-render

```
function AddButton() {  
  const [count, setCount] = useState(0);  
  
  return (  
    <button onClick={() => setCount(c => c + 1)}>  
      Added {count}  
    </button>  
  );  
}
```

- `useState(0)` hands back two things: the value now, and a function that asks for a new one.
- The setter does not assign. It requests another render, and React runs your function again with the new value.
- An ordinary `let` inside the component would be reset every render — that is exactly what state exists to fix.
- `c => c + 1` is given the pending previous value, which is the version that is always correct.

State is the answer to: what does this component need to remember between one render and the next?

- Call `useState` at the top of the component. Never inside an `if`, a loop, or a nested function.

EVENTS

Pass the handler, do not call it

```
onClick={handleClick}
```

Correct. You hand React the function. React calls it when the click happens.

```
onClick={handleClick() }
```

Runs the moment the component renders, and hands React whatever it returned. A setter here loops forever.

```
onClick={() => setCount(c => c + 1)}
```

Correct, and the common shape. The arrow function is the handler; it is defined, not called.

onClick takes a function. The extra pair of parentheses is the difference between giving someone a recipe and cooking the meal.

"Too many re-renders" in the console almost always means a setter ran during render rather than in a handler.

WHERE STATE LIVES

Two instances, two memories

`<Counter />`
count: 2

`<Counter />`
count: 0

One definition. Each rendered instance gets its own `useState`, so clicking one does nothing to the other.

`<Pantry />`
count: 2

`<Counter />`

`<Counter />`

When both need the same number, it moves up to the parent they share.

```
function Pantry() {  
  const [count, setCount] = useState(0);  
  const add = () => setCount(c => c + 1);  
  return <> <Counter count={count} onAdd={add} />  
    <Counter count={count} onAdd={add} /> </>;  
}
```

Data goes down as props. A change goes up as a call. This is the whole data-flow model — there is no third direction.

FORMS

A controlled input

```
function ProductNameField() {
  const [name, setName] = useState('Apples');

  return (
    <>
      <label htmlFor="product-name">Product name</label>
      <input id="product-name"
        value={name}
        onChange={e => setName(e.target.value)} />
      <p>Shopping for: {name}</p>
    </>
  );
}
```

- `value={name}` makes state the single source of truth for what is in the box.
- That alone would make the field read-only — typing would change nothing.
- `onChange` closes the loop: keystroke, setter, re-render, new value in the box.
- `e.target.value` is the text in the input right now.
- `htmlFor` matches the input's id. It is why clicking the label focuses the field — and in session 11 it is how the tests find it.

Everything that depends on the field updates on the same keystroke, because they all read the same piece of state.

Where should this state live?

1 Does anything change it?

If not, it is not state. A constant, or a prop from above, or just a value in the file.

2 Can you work it out?

If it can be computed from other state or props, compute it during render. Do not store it twice.

3 Who needs to read it?

Put it in the nearest component that contains everyone who reads it. Not higher.

4 Pass a function down

A child never reaches up. It calls the callback it was given and the parent decides what happens.

Two copies of the same fact in two places will disagree. That is the bug from slide three, back again in a new costume.

WHAT WILL GO WRONG

Five mistakes we will all make

```
class="flex"
```

className. The console says nothing useful; the styling simply does not appear.

```
onClick={handleClick() }
```

Runs at render. With a setter inside, the tab freezes: "Too many re-renders".

```
let count = 0; count++
```

Resets on every render. Nothing on screen moves. Use useState.

```
key={index}
```

Looks fine until items move or get deleted, then state attaches to the wrong row.

```
value={name} with no onChange
```

The field will not accept typing. It is behaving exactly as written.

Four of these five produce no error at all. Read the page, not just the console.

IN CLASS

The workshop

One file first

`00-micro.html`

React and Tailwind from a CDN, opened with Live Server exactly like HW3. One heading, one change, no npm. Proves React is a library you add to a page, not a different kind of website.

Then the real setup

`npm ci · npm run dev`

Vite gives you JSX and reloads on save. Node 22.12 or newer. Install it before Tuesday if you can — class time is better spent on React than on npm.

Eleven drills

`src/drills.jsx`

One idea each, three to ten minutes, with a Check result button that reads the rendered page rather than your class names. Worked answers one click away. Nothing here is graded.

Predict, change one thing, save, check, explain it to the person next to you. Same loop as the HW3 drills.

One React element in an HTML page

THE REQUEST

Change the heading text to Hello, React!
Then try text-3xl in place of text-xl.

WHAT IT DEMONSTRATES

`createElement` describes a tag, its properties and its content. `createRoot` gives React a place to render.

KEY CODE

```
const heading = createElement(
  'h2',
  { className: 'text-xl font-bold' },
  'Hello, React!'
);
createRoot(
  document.getElementById('root')
).render(heading);
```

EXPECTED RESULT

The heading changes and the check says PASS. The class change makes the heading larger.

The local workshop setup

Vite runs the JSX workshop

Use the supplied react folder. It requires Node 22.12 or newer.

Install the locked dependencies, start Vite and open its Local URL.

Edit src/drills.jsx for drills 1–10.

```
cd inclass/wk5/react  
node --version  
npm --version  
npm ci  
npm run dev
```

Checkpoint: saving Drill1 updates the browser preview.

Vite turns source files into a fast browser preview

It is the development and build tool around React—not React itself.



React describes the UI. Vite gets that code into the browser quickly.

Vite creates the production files—then steps out

For a typical client-side React app, Vite is a development dependency and build-time tool.

BEFORE DEPLOYMENT

```
npm run build
```

Vite transforms JSX, bundles modules, minifies assets, and writes deployable files to dist/.



DEPLOY

IN PRODUCTION

Server or CDN → browser

The host serves HTML, JavaScript, CSS, and assets. The browser runs the built application.

Vite is not running here

vite preview is for checking the production build locally—not for serving the real production site.

Usually a devDependency does not mean “development only.” Vite also performs the production build.

JSX describes the heading

THE REQUEST

In Drill1, replace Hello, HTML! with Hello, React! Save and check the preview.

KEY CODE

```
export function Drill1() {  
  return <h2>Hello, React!</h2>;  
}
```

WHAT IT DEMONSTRATES

JSX expresses markup inside JavaScript. A component function returns the UI that React renders.

EXPECTED RESULT

The preview shows Hello, React! and the check passes.

Braces evaluate a JavaScript expression

THE REQUEST

Replace the literal word product inside the paragraph with {product}.

KEY CODE

```
const product = 'Apples';  
  
return <p>{product}</p>;
```

WHAT IT DEMONSTRATES

Text between tags stays literal. Braces insert the value of a JavaScript expression into JSX.

EXPECTED RESULT

The paragraph displays Apples. Changing the variable to Pears changes the output.

Tailwind utilities still control layout

THE REQUEST

Add classes for a flex row, vertical centering and space between the two items.

KEY CODE

```
<div className=  
  "flex items-center justify-between">  
  <strong>Pantry</strong>  
  <span>3 items</span>  
</div>
```

WHAT IT DEMONSTRATES

React uses className for CSS classes. The Tailwind utilities retain the same meaning as in HTML.

EXPECTED RESULT

Pantry sits at the left and 3 items at the right, vertically centered.

Props customize a reusable component

THE REQUEST

Inside Product, display name in the h3.
Leave the two component calls unchanged.

KEY CODE

```
function Product({ name }) {  
  return <h3>{name}</h3>;  
}
```

```
<Product name="Apples" />  
<Product name="Rice" />
```

WHAT IT DEMONSTRATES

Props supply inputs to a component. One function definition can render different content for each call.

EXPECTED RESULT

The same Product definition displays Apples and Rice.

An array supplies the repeated markup

THE REQUEST

Replace the placeholder li with `products.map(...)`. Read each name and use its ID as the key.

WHAT IT DEMONSTRATES

`map` produces one element per item. Stable keys let React associate each element with its data.

KEY CODE

```
{products.map(product => (  
  <li  
    key={product.id}  
    className="rounded bg-slate-100 p-4"  
  >  
    {product.name}  
  </li>  
))}
```

EXPECTED RESULT

Apples, Rice and Beans appear in order. The retained grid uses one column below 768px and three at wider sizes.

A boolean chooses the displayed text

THE REQUEST

In Freshness, replace Status with the conditional expression shown here.

KEY CODE

```
{expired ? 'Expired' : 'Fresh'}  
  
<Freshness expired={true} />  
<Freshness expired={false} />
```

WHAT IT DEMONSTRATES

The ternary operator chooses one expression when the condition is truthy and the other when it is falsy.

EXPECTED RESULT

The first badge says Expired. The second says Fresh.

A click requests a state update

THE REQUEST

Add `setCount(c => c + 1)` inside `handleClick`. Keep the existing `onClick` connection.

WHAT IT DEMONSTRATES

The setter requests another render. React runs the component with the updated state and updates the affected DOM.

KEY CODE

```
const [count, setCount] = useState(0);

function handleClick() {
  setCount(c => c + 1);
}

<button onClick={handleClick}>
  Count: {count}
</button>
```

EXPECTED RESULT

Reset the preview, then click twice. The button shows Count: 2.

Each Counter instance owns its state

THE REQUEST

Render a second `<Counter />` inside `Drill8`.
The supplied `Counter` already works.

WHAT IT DEMONSTRATES

Two instances of the same component have separate state. Reusing a function does not share its count.

KEY CODE

```
return (  
  <div className="flex gap-4">  
    <Counter />  
    <Counter />  
  </div>  
);
```

EXPECTED RESULT

After reset, click the first twice and the second once. The counts are 2 and 1.

The parent owns the shared count

THE REQUEST

In SharedCounter, connect the button's onClick to onAdd. Keep state in Drill9.

WHAT IT DEMONSTRATES

The parent passes the same count and callback to both children. Either child can request an update to that shared state.

KEY CODE

```
function SharedCounter({ count, onAdd }) {  
  return (  
    <button onClick={onAdd}>  
      Count: {count}  
    </button>  
  );  
}
```

EXPECTED RESULT

After reset, click each button once. Both buttons display Count: 2.

State controls the input value

THE REQUEST

Replace the empty onChange handler with `e => setName(e.target.value)`.

WHAT IT DEMONSTRATES

The input reads its value from state. Typing sends the next string back into that state, which also updates the paragraph.

KEY CODE

```
<input
  id="product-name"
  value={name}
  onChange={e => setName(e.target.value)}
/>
<p>Shopping for: {name}</p>
```

EXPECTED RESULT

Replace Apples with Rice. The paragraph immediately shows Shopping for: Rice.

A PantryCard combines the pieces

THE REQUEST

Control Input with value and onChange.
Pass onAdd as the Button's onClick prop.

WHAT IT DEMONSTRATES

The parent owns name and count.
PantryCard combines shadcn/ui
components with your props and behavior.
Tailwind controls layout.

KEY CODE

```
<Input  
  id="pantry-name"  
  value={name}  
  onChange={e => setName(e.target.value)}  
>
```

```
<Button variant="secondary"  
  onClick={onAdd}>Add one</Button>
```

EXPECTED RESULT

Rename to Rice and click Add one twice. The card shows Rice and Quantity: 2.

Component tests describe visible behavior

The components already work

Edit `src/pantry.test.jsx` to complete the four tests.
Leave `src/pantry.jsx` unchanged.

Open a second terminal and start the watcher:

`npm run test:unit:watch`

The four starters fail intentionally. Read each failure, complete the test and save.

A test finds the heading by its role

THE REQUEST

Replace CHANGE ME with Pantry in the heading assertion.

KEY CODE

```
render(<ProductHeading />);

expect(
  screen.getByRole('heading', { level: 2 })
).toHaveTextContent('Pantry');
```

WHAT IT DEMONSTRATES

render mounts the component in a test DOM. A role query describes the visible semantic element that the test expects.

EXPECTED RESULT

Drill 12 passes. A font-size class change can preserve the test, while a heading-level change breaks it.

Assertions describe the effect of props

THE REQUEST

Assert that `name="Rice"` produces Rice and `expired={true}` produces Expired.

WHAT IT DEMONSTRATES

A test can describe the output contract of a component for a particular set of inputs.

KEY CODE

```
render(<Product name="Rice"
  expired={true} />);

expect(
  screen.getByRole(
    'heading', { level: 3 })
).toHaveTextContent('Rice');
expect(screen.getByText('Expired'))
  .toBeInTheDocument();
```

EXPECTED RESULT

Both assertions pass: the heading says Rice and the Expired badge exists.

A test waits for each click

THE REQUEST

Add two awaited `user.click(...)` calls before the existing `Added 2` assertion.

WHAT IT DEMONSTRATES

Arrange the component, perform the actions, then assert the result. Await each interaction before checking the updated UI.

KEY CODE

```
const user = userEvent.setup();
render(<AddButton />);

await user.click(screen.getByRole('button'));
await user.click(screen.getByRole('button'));

expect(screen.getByRole('button'))
  .toHaveTextContent('Added 2');
```

EXPECTED RESULT

Two completed clicks produce `Added 2`, so drill 14 passes.

A typing test uses the input label

THE REQUEST

Find Product name through its label. Clear the field, then type Oats with awaited actions.

WHAT IT DEMONSTRATES

The label connects the query to an accessible input. Clearing first replaces the existing text because `user.type` appends.

KEY CODE

```
const field =  
  screen.getByLabelText('Product name');  
await user.clear(field);  
await user.type(field, 'Oats');  
  
expect(field).toHaveValue('Oats');  
expect(screen.getByText('Shopping for: Oats'))  
  .toBeInTheDocument();
```

EXPECTED RESULT

The input contains Oats and the paragraph says Shopping for: Oats. All four tests pass.

Explaining the complete example

Component behavior

Where do name and count live?

Which props does PantryCard receive?

How does a button click change the next render?

Evidence of understanding

Change a prop or data item and predict the output.

Write one additional test, such as the Fresh badge, a third click or an empty field.

A passing check supports the result. Your explanation shows how the code produces it.

HOMWORK 4

React components, due Friday 10/2

- Finish drills 1 to 11 and commit them. If you did them in class, that part is already done.
- Then one extension: a component of your own on the pantry page — a prop, a piece of state, used twice.
- Deliberately small. Local state only — no fetching, no router, no backend. About an hour.
- Homework 5 the week after adds your own component tests, which is session 11 and the last four drills.

Also due Friday 9/25

Homework 3 — flexbox, grid and breakpoints — and your project proposal. Read the week 5 page for the reading: the React docs, plus Fluent React chapter 3 through the RPI library.

Bring a laptop with Node installed on Tuesday. Everything this week happens in VS Code and a browser.